

Changes in Direction

The final project is quite similar to what was proposed in the Stage 1 proposal submission. This is due to the careful planning that was conducted at the beginning of the project. While many ideas were proposed in Stage 0 and Stage 1, we analyzed each idea in a fair manner and quickly decided on the project that we thought would be the best in terms of team member interest and project requirements. In Stage 1, we had already created an initial database design that wasn't actually required until a later stage of the project which helped with discussions. We iterated on this database design with multiple revisions in Stage 2 until we reached the final version that the project implemented (with some very minor changes).

While the final project is very similar to our initial proposal (by ~90-95%), there were some minor changes. There were some superfluous features that were not pursued while some others were added instead. This is due to the developers having an increased understanding of the fundamentals required to build the application. For the features we removed, we severely underestimated the difficulty required to implement them and thus focused more on improving the core functionality of the application.

- An advanced visualization feature where a diagram of a human would be shown. The idea was to give the user the ability to choose muscles to filter their searches on by clicking on the muscles on the diagram. Since we proposed multiple advanced features and the other features interested the project members more, we decided to focus on the other advanced features, instead.
- A calorie counting/tracking feature which would sync up to a user's apple watch and provide detailed workout information while the user was working out. We decided to drop this idea due to the complexity that would have been involved in making the feature work because it required integration with an external application programming interface and external hardware. This was an optional advanced feature that wasn't required for our project.
- A historical metrics/tracking feature. We initially suggested a possible metrics feature that would track previous workout information and display the information to the user with some graphs. We decided to not fully implement this idea and to instead focus on other related features which we believed would improve the user experience better. While the application does not provide historical metrics to the user with graphs, the application instead tracks other metrics such as the duration of workouts, the previous set information for how each exercise was previously performed (to automatically populate set information when creating new workouts so the user does not have to remember this information), and tracks the amount of time each exercise has been performed. With the tracking of these metrics instead, we believe the application is improved due to its increased usability in comparison to the simple graphs feature we would have provided.

What Usefulness we Achieved/Failed to Achieve

We were able to achieve most if not all of the features that were proposed to make the application as useful as possible. Firstly, we implemented the creation, modification, reading, and updating of exercise data. We did so by making the exercise pages modifiable if the user is the owner of the exercise.

Secondly, we implemented the advanced exercise search functionality that was proposed in the previous stage. This feature worked by filtering exercises the user has access to by muscleId and keyword search and finally ordered the results listed on the page by best match. We additionally implemented a simpler search bar which filtered the workout templates listed on the Workouts (workouts list) page with the keywords specified in the search bar.

Next, we implemented the creation, modification, reading, and updating of the workout data through several features. With a button, users could create new empty workouts on the Workouts (workouts list) page. Additionally, each of the displayed cards (which could be clicked to navigate the user towards each individual Workout page) contained an “x” button that could be clicked to delete the workout template and its associated workout contents. On each individual workout page, users could view the workout that was displayed with a repeatable read transaction (to ensure the different aspects of the page were in sync when it was built to prevent confusion for the user). The page displayed the name, each exercise’s name in the specified order on the page, all of the sets that were related to each exercise in proper order under each exercise, and buttons could be used to add new exercises and sets. We also implemented an advanced query here which would suggest an exercise to add to the workout based on the exercises the workout already included. The user could store their modifications to the workout with a button which would invoke a complicated stored procedure that would update all associated data. It would update what exercises were associated with the workout template and in what order as well as what sets were associated with each exercise and in what order. If new sets needed to be created, they would be created and associated properly. The lastDate parameter was also updated based on the current date and time of the server. The page additionally had a working timer which was synced up to another stored procedure which would log all of the previously mentioned data as well as the timer’s duration and would update the number of times each exercise in the workout has been completed. Both stored procedures were very complicated because they modified four tables and required multiple loops to properly iterate through the exercise and set information.

We additionally implemented a Dashboard page which would display the three most recent workouts the user has performed so they can workout in a faster, more user friendly, manner. The cards would additionally display the duration of the workout the

last time it was performed. The dashboard also displays the total amount of exercises performed by the user to give a sense of progress to boost motivation.

In terms of security, we utilized nextjs middleware and HTTP session cookies to authenticate user sessions and *protect the website's private routes* like `/dashboard`, `/workouts` and `/exercises`. The middleware layer intercepts user requests at the server level where it reads an 'auth' object containing the valid session information, authenticated users will be allowed access to website private features while unauthenticated users are always redirected to the `/login` route. Once logged in a user will automatically be redirected to their `/dashboard` and will find they are no longer able to access the login route as they are already authenticated. We've also implemented signout functionality to compliment the authentication workout.

Login and signup pages require 2 separate SQL queries, the login query is fairly trivial and asserts a password match on a valid username in the Users table, a simple oneliner. The *signup query mimics a SQL transaction* although it was technically implemented as a GCP Procedure, it checks whether or not a username already exists in the database and will either fail or succeed depending on the provided username. If successful, the transaction returns a message `{ success: 0 }` following C style convention, otherwise an error code is returned, for example `{ success: 1 }`. You might imagine we can up the complexity of the response object to account for potentially several error codes (1,2,3,4...) and use a switch-case statement on the client to thoroughly parse out any database errors we might encounter, thus helping maintain ACID compliance while signalling possible errors.

The calorie counting, historical metrics, and advanced visualization features that were mentioned earlier were not implemented for reasons that were outlined in a previous section. Since we proposed multiple advanced features (to decide on later), we actually did not need to implement these features because we implemented two of the proposed advanced features: advanced security and the exercise suggesting button mentioned in the above Workout page description.

How the Schema and Source of Data Changed

Initially three possible sources were identified as potential sources of data to fill the database tables with. We decided to use only one of the identified sources, the Gym Exercise Dataset on Kaggle, because it contained most of the data we required. With the data that wasn't provided we were able to create a suitable amount of data with Excel commands, manually inputting data, and as a last resort generating data that we were unable to find a suitable source for.

Due to the careful planning we implemented in Stage 3, we only had to make a single small change to the schema to implement the final project. In the WorkoutTemplates table, we added a new parameter called "lastDuration" which was an

integer that was null by default. This parameter was added to support the creation of a workout timer which the user could use to time the duration of their workouts. The parameter would be set to null if the user had never performed the workout before. The introduction of the workout timer was not required but it was a new feature we decided to add because we believe it increases the usability of the application.

How the ER Diagram, and Table Implementations Changed

The final ER diagram only had two small changes. We decided to add a feature that allows users to track the duration of workouts, this required us to add another attribute under the WorkoutTemplates entity called lastDuration to store the workout duration in seconds. We believe this makes our final design more suitable for the user because the addition of the attribute increases the usability of the application for the user. Users now have an understanding of how long a workout will likely take (based on how long it previously took them to complete it) so they'll better be able to schedule their workouts into their busy schedules.

We additionally fixed a small mistake that was in our original ER diagram (but was fixed in our schema). In our original ER diagram we accidentally listed "exerciseld" and "setld" as a composite primary key under the ExerciseLog entity (both attributes were underlined). This was a typo, however, ExerciseLog actually required a composite primary key of "exerciseld" and "userId" so the underlined setld attribute was correctly replaced with the underlined userId attribute in the final diagram. This change is a more suitable design for our application because ExerciseLog needs to explicitly store logs separately for each user, which it is now able to do because the userId is now part of its composite primary key (this was correct in the Stage 3 schema).

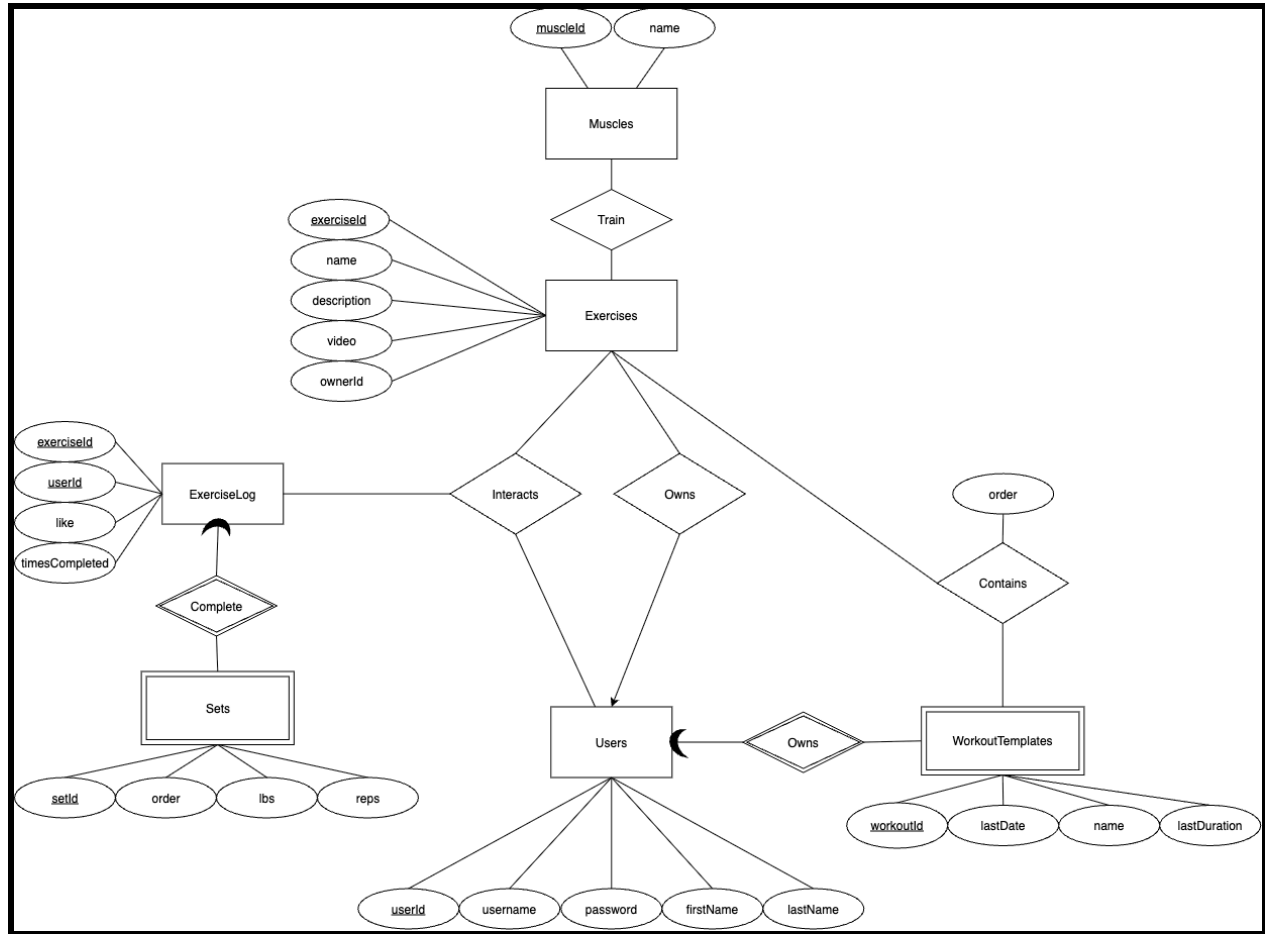


Figure 1: Final ER Diagram

The only change that was made to our implemented database tables and schema was the addition of the prior mentioned lastDuration parameter to the WorkoutTemplates table. The addition of this field supported the addition of a workout timer to track workout duration which increased the usability of the application.

We believe our final schema and ER diagram are a more suitable design for the application because they improve the user experience of the application and fix a small typo that was in the previous ER diagram that would have been a disastrous mistake in the final application (if the mistake was implemented users share and thus be able to modify each other's ExerciseLogs which would be a mistake).

Added/Removed Functionalities

We added a workout timer which can be used to track the duration of workouts. This required creating a timer in the application code which would update the time that has passed in real time. The timer required a start, pause, and finish buttons to work properly. We added this functionality because we believe this improved the usability of

the application since users will be able to schedule workouts into their busy schedules in an easier manner since the duration of the workout is calculated and stored.

We additionally added a search bar to the Workouts (workouts list) page which the user can use to search for workout templates to use to workout with. We added this feature so that users who have a large amount of workout templates created can more easily find the workout template that they want to use for their workout. This makes the application easier to use.

We added a dashboard page which displays the three most recent workouts. This feature makes it easier for a user to repeat recently completed workouts because they do not actually have to search for the workout they want to perform on the Workouts (workouts list) page. We believe this feature makes the application easier to use. The dashboard page also displays the amount of exercises completed to give the user a visualization of the progress they have made.

How Advanced Database Programs Complement Application

When building the individual Workout page, we use a repeatable read transaction which contains multiple queries which read from the database. We decided to use a repeatable read transaction so the read locks are maintained throughout the course of the transaction. We made the decision to hold onto the locks because we did not want data to be written in between the queries that would result in the page containing a mix of data (that would be a combination of multiple workout instances) which would confuse the user.

We implemented two stored procedures in the application. The user could store their modifications to the workout with a button which would invoke a complicated stored procedure that would update all associated data. It would update what exercises were associated with the workout template and in what order as well as what sets were associated with each exercise and in what order. If new sets needed to be created, they would be created and associated properly. The `lastDate` parameter was also updated based on the current date and time of the server. The page additionally had a working timer which was synced up to another stored procedure which would log all of the previously mentioned data as well as the timer's duration and would update the number of times each exercise in the workout has been completed. Both stored procedures were very complicated because they modified four tables and required multiple loops to properly iterate through the exercise and set information.

In our schema we defined appropriate primary keys and foreign keys which were used to perform assertions that would satisfy the application's constraints. For example, the `ExerciseLog` table uses a composite primary key of `userId` and `exerciseId` to ensure that the exercise data is logged separately for each exercise for a given user. In doing so, users are unable to mistakenly modify each other's exercise logs. We additionally use a composite primary key (made up of `userId` and `workoutId`) to ensure that each

user has their own WorkoutTemplates associated with them and since it is a weak entity relationship, the WorkoutTemplates are deleted if the user is deleted.

Technical Challenges

Lucas found creating and integrating the stored procedures to be a very challenging aspect of the project. The stored procedures were very difficult to test, integrate, and write. He ran into a lot of issues when it came to understanding how the data types were inside of the json data when it was being communicated back and forth between the stored procedure and application code. It was difficult to troubleshoot where issues were occurring. He spent several hours trying to figure out why the lastDate parameter was being incorrectly parsed into a DATETIME column (with many different iterations on the manner of parsing and processing) only to later discover that the data was being parsed and stored correctly all along, the issue was that the application code was mishandling a datatype. We believe the application was incorrectly converting the received date object was unknowingly being converting into the wrong timezone. Lucas' advice would be to write test code which tests the application code to catch application specific errors, which was something our group ran out of time to do. Lucas would additionally recommend setting aside time to find tools that can help with application development debugging. Maybe he would've discovered the bug sooner if he better understood how the datatypes were being converted when being sent back and forth between the client and server.

Rami found that implementing user oriented CRUD features into the exercise page for each user and each individual exercise to be a large challenge. The "create exercise" page was the easiest to implement, as it was simply inserting user made text into the database while assigning their user id as the ownerId. Where things got tricky was with the update and delete options for the user. Updating an exercise a user owns means that at the page's creation, the user would have to be checked with the owner id of the exercise before anything happens, or else the user would have been able to edit exercises that were not their own. This same issue would apply to the deletion of exercises. The main issue that was faced though was not the creation of the code, but was properly using the current userId and tying it to all of this, while constantly referencing the current sessions userId, which needed to be retrieved from the authentication token. Getting this token to communicate with the queries was not an issue at first, but when building the final version, there were dozens of build errors that were caused by type conflicts from the authentication token to the code used in the exercise CRUD features. Fixing this resorted to nearly the entirety of the exercises CRUD related pages to be rewritten and hours of time wasted where other features could have been implemented. The advice that Rami gives is to extensively test one's

code in chunks as one makes it. This would have told Rami from early on that there were errors in his code and between the types, saving him hours of time.

Sean encountered a series of complex challenges while developing the interactive workout editor UI, which required connecting multiple frontend components with the backend database. One of the most significant hurdles was ensuring full-stack feature correctness across multiple users. A bug was reported where a user could create an empty workout but would then be met with a "Workout Not Found" error. The initial investigation suggested a hardcoded user ID on the workout page itself, but a thorough code review proved this false. The root cause was traced back one step earlier in the user flow: the `/api/createEmptyWorkout` API endpoint. It was using a hardcoded `userId=1` during the creation process. This meant the workout was created for the wrong user, so when the correct user was redirected to the page, the database query correctly found no matching workout for their session ID. The key learning was to trace the entire data lifecycle, from creation to retrieval, to find the true source of an error. The fix involved replacing the hardcoded ID in the creation endpoint with the dynamic `userId` from the secure server-side session. Sean's advice for future teams would be knowing how to navigate through your file directory for the project and search for certain keywords in your repo. In his case, searching the files for `userId = 1` to see where the value was hardcoded was essential in making the updates needed to fix that bug.

When creating the middleware authentication layer Serge ran into an issue configuring the `Credentials` database adapter to comply with both the *client javascript runtime* as well as the *middleware edge runtime*. The problem was the database `mysql2` adapter used by clients to authenticate to the service uses libraries not available in the lighter, streamlined edge runtime where the middleware layer exists. After reading the [Auth.js documentation](#), Serge implemented a split-config Authjs authentication configuration, pairing a lightweight authentication context to run the middleware layer in the edge runtime with a separate fully featured authentication context to run clientside using the client's Javascript engine. Essentially, when pairing Authjs version 5 with server side rendered web-frameworks like nextjs, make sure to read the Authjs documentation on configuring middleware in *edge runtimes* as many database adapters still have issues working in lightweight edge runtime environments.

Did Other Things Change

We already identified all things that changed between the final application and original proposal so there is not anything to discuss in this section.

Possible Improvements/Future Work

We would recommend writing test code that tests the major aspects of the project with the test code coverage determined by the complexity of the component and the risk

involved if the component fails. We would additionally recommend adding a page to the application which displays historical workout metrics. This would require modifying the database to track previous workouts over time (which is different from what we do where we just track the most recent date a workout was performed and the number of times each exercise was performed). We additionally would recommend the addition of a calorie counting/tracking feature which would sync up to a user's apple watch and provide detailed workout information while the user was working out. We believe this feature would make the application appeal to a wider audience.

As *fun* as it was to write raw SQL queries to communicate back and forth with our database using the mysql2 driver, modern day websites have largely ditched this in favor of ORMs (Object Relational Models). Many modern ORMs closely mimic SQL syntax while cleanly preserving the database schema across web applications, this yields several benefits. One, static type checking enforces the database schema across the application so we don't encounter any arbitrary type inference errors when querying the database, honestly a headache to deal with. Two, any changes to the schema will immediately highlight breaking changes in the clientside code, allowing for fast iteration with less overhead. Three, implementing advanced SQL queries like procedures and triggers becomes trivial and can be done clientside in some sort of schema or database folder, once finished advanced queries can be exported, used, and clearly referenced throughout the entire application. For these reasons we highly suggest *allowing* the use of ORMs in future sessions of CS 411 as an added extra credit feature.

Final Division of Labor

We collaborated with each other and divided up tasks based on each other's strengths and weaknesses as well as their interest in the given subject matter. In doing so, each team member collaborated well with each other and performed their assigned tasks in a sufficient manner to complete the project on time.

Lucas wrote and integrated the stored procedures which were a major part of the back-end functionality required for the Workout page. He wrote most of the back-end functionality for the Workouts (workout listing) page. He additionally added creating and deleting of Workout Template cards to the page. Displaying the page with the user's Workout Templates required a transaction which performed two reads at an isolation level of repeatable read. He was mainly in charge of the application's design (this included doing a large amount of the work for the database design, schema, ER diagram, etc), did a large amount of the programming and writing of the reports, and was the project lead.

Sergiu mainly *implemented the login/account creation features as well as protecting the website private routes* like '/dashboard', '/exercises', and '/workouts'. As an added note, Serge *configured the Cloud SQL mySQL database hosting on GCP and automated the website hosting deployment pipeline* to trigger build and deployment

procedures on changes to the GitHub repo 'prod' branch, thus hosting both the database and the website in GCP cloud infrastructure - an extra credit option if we're not mistaken. Finally, to stay on budget, Serge *configured custom GCP billing notifications* to signal to the team when we'd neared our allocated cloud budget of \$5 a month, something we never exceeded.

Rami created the dashboard and exercise pages, and the workout timer, along with implementing the user design and "flow" of data throughout the website. These are front-end-focused and designed to enhance the user experience. He implemented the advanced query related to the exercise page, while also hitting all CRUD requirements. Rami helped design the website's interface and visual look, hoping to ensure users can easily access what they want to access, while having buttons in the most user-expected spots. He also helped work on the server to client pipeline from the front end perspective, allowing the client to properly interpret what is being sent by the server. Rami also implemented page to page connections, allowing for simpler flows between frontend pages to simplify the user's experience (for example, when a workout is done, the user will be sent to the dashboard page to see their progress, and when a new user is created, that dashboard will give them a redirect to the workouts page to start creating exercises). These changes aid the user in making the website more readable and usable in day to day settings. Also made the website more visually uniform across the board.

Sean was responsible for developing the interactive workout editor UI, which served as the primary interface for the application's core CRUD operations. This involved building a complex React component using hooks like useState and useCallback to manage the client-side state for the workout's name, exercises, and sets. He integrated the dnd-kit library to implement drag-and-drop functionality for reordering both exercises and their individual sets. He connected this stateful UI to the backend by implementing the fetch logic for the "Save Workout" and "Finish Workout" buttons, which packaged the data to be sent to the stored procedures. Furthermore, he developed the "Add Exercise" feature, which included creating a dedicated API route (/api/exercises) to read from the database and a dynamic search modal that allowed users to find and add new exercises to their current workout. Finally, he implemented the "Suggest an Exercise" creative feature, which involved creating a new API endpoint to execute his advanced SQL query from Stage 3. This query intelligently suggests a random exercise based on a user's completion history, avoiding repetition and providing a "smart" user experience. He also played a key role in debugging multi-user authentication issues and resolving production build errors.

Stage 0 and 1

For Stage 0 and Stage 1 we only lost points because we forgot to make a release for the code (we had it tagged though). We went back and made releases for those stages:

<https://github.com/cs411-alawini/su25-cs411-team007-FutureLegends/releases/tag/stage.0>

<https://github.com/cs411-alawini/su25-cs411-team007-FutureLegends/releases/tag/stage.1>